



UNIVERSIDAD AUTONOMA DE SANTO DOMINGO(UASD)

TEMA:

MVC en el Desarrollo Web Moderno: Origen, Evolución y Uso Actual

Autor: Luis Enrique Leocadio Aquino

Matricula: 100457430

Asignatura: Lab. Programación III

Profesor: Radhames Silverio González

Introducción

En el desarrollo de software moderno, organizar el código y mantener una separación clara de responsabilidades constituye un pilar indispensable para asegurar la escalabilidad, legibilidad y testabilidad de los sistemas corporativos y plataformas web. El patrón arquitectónico **Modelo-Vista-Controlador (MVC)** ha permanecido como uno de los enfoques conceptuales más influyentes y adoptados a nivel mundial. Aunque nació en una era previa a la existencia del Internet comercial, su estructura lógica demostró ser tan maleable y robusta que acabó convirtiéndose en el núcleo de la ingeniería web moderna.

Este documento presenta una investigación estructurada en torno a MVC, detallando su definición formal, sus raíces históricas en los laboratorios Xerox PARC, el rol operativo de cada uno de sus tres componentes esenciales y un balance objetivo de sus ventajas y limitaciones. Asimismo, analiza los mecanismos mediante los cuales los principales entornos del desarrollo actual (como Laravel, Django y Spring) implementan este paradigma arquitectónico, culminando con un ejemplo ilustrativo guiado y una conclusión de carácter metodológico bajo el formato académico estándar (APA).

Desarrollo de la Investigación

¿Qué es el patrón MVC?

El Modelo-Vista-Controlador (MVC) es un **patrón de arquitectura de software** enfocado en la división del código fuente en tres capas o componentes autónomos e interconectados de manera lógica. Su fin primordial es la **separación de conceptos (Separation of Concerns, SoC)**. Al aislar las reglas lógicas del negocio, los datos del sistema y los elementos de interfaz, se asegura que las variaciones aplicadas sobre el apartado gráfico no comprometan el comportamiento interno de los datos, reduciendo drásticamente el acoplamiento y facilitando el mantenimiento preventivo y correctivo del software.

1.- Origen del modelo MVC y para qué fue creado

El patrón MVC cuenta con casi cinco décadas de historia. Fue ideado y desarrollado por el científico de la computación de origen noruego **Trygve Reenskaug** en el año 1978, durante su estancia de investigación en el célebre centro **Xerox PARC (Palo Alto Research Center)**. Su materialización se incluyó como parte medular del entorno interactivo y lenguaje de programación **Smalltalk-80**.

Es crucial matizar que MVC no nació con la finalidad de gestionar aplicaciones web, dado que la Web no existía en dicho periodo histórico. Su objetivo primario consistía en proveer una solución al desafío arquitectónico que representaban las nacientes **Interfaces Gráficas de Usuario (GUI)** de escritorio. Frente a las tradicionales terminales de línea de comandos, las nuevas interfaces visuales integraban

ventanas, menús y ratones. Reenskaug ideó MVC para que los usuarios logaran interactuar fluidamente con réplicas conceptuales de su información, permitiendo visualizar los mismos datos subyacentes a través de distintas pantallas concurrentes e interactivas sin corromper la consistencia lógica de la información del sistema.

2.- Explicación detallada de cada componente

- **Modelo (Model):** Es el corazón operativo de la aplicación. Representa directamente el conjunto de datos de trabajo y gestiona formalmente la **lógica de negocio** (reglas corporativas, validaciones restrictivas de campos, cálculos y operaciones algorítmicas). Es plenamente autosuficiente: no conoce en absoluto la existencia de las vistas ni de las interfaces de usuario que consumen su estado.
- **Vista (View):** Constituye la **capa de presentación visual** o interfaz gráfica dirigida al usuario. Su rol es transformar las variables provistas por el modelo en interfaces gráficas asimilables (pantallas HTML, plantillas dinámicas XML o flujos estructurados JSON). En la arquitectura tradicional, la vista permanece pasiva, actuando meramente como reflejo visual del estado del modelo.
- **Controlador (Controller):** Desempeña el papel de **mediador y canalizador operativo** del flujo lógico. Captura activamente las acciones solicitadas por el usuario final (tales como pulsaciones de teclado, clics de mouse o peticiones de enrutamiento web HTTP), ejecuta las transformaciones correspondientes invocando los métodos específicos del modelo, y determina cuál es la vista indicada que debe renderizarse para notificar el nuevo estado al usuario.

3.- Ventajas y desventajas del uso de MVC

Ventajas de Implementación	Desventajas y Desafíos
Desacoplamiento Estricto: Facilita que diseñadores front-end e ingenieros back-end trabajen en paralelo de manera limpia.	Curva de Aprendizaje Compleja: Exige una disciplina de diseño estricta y mayor cantidad de archivos en proyectos iniciales.
Múltiples Vistas Simultáneas: Un único modelo lógico de datos puede alimentar una web HTML, una API móvil y un reporte PDF.	Sobrecarga en Controladores: Si no se vigila, el controlador puede acumular código de negocio indebido ('Fat Controllers').
Facilidad de Testeo Automatizado: Al estar aislada de la interfaz gráfica, la lógica del Modelo admite pruebas unitarias precisas.	Ineficiencia en Proyectos Mínimos: Para herramientas hiper-simples o páginas estáticas, MVC introduce capas redundantes.

4.- ¿Por qué MVC sigue siendo importante en la actualidad?

Aunque en la actualidad coexisten múltiples paradigmas alternativos, MVC conserva una vigencia incuestionable en la ingeniería de software moderna por tres pilares elementales: establece un estándar universal comprensible y predecible para equipos globales de ingenieros; actúa como la base pedagógica para entender variantes arquitectónicas avanzadas como MVVM (Model-View-ViewModel) en arquitecturas SPA, y se complementa nativamente con los ciclos automatizados de integración continua (CI/CD) gracias al aislamiento de sus pruebas unitarias.

5- Uso actual en frameworks modernos y comparación

En sus inicios, MVC operaba bajo entornos con memoria compartida local donde el modelo notificaba a la vista en tiempo real vía patrones estructurados como Observer. Al trasladarse a la arquitectura de la red Internet, el patrón debió mutar sustancialmente para alinearse con la naturaleza **sin estado (stateless)** e intermitente del protocolo HTTP (ciclo petición-respuesta).

Hoy en día, el panorama de frameworks modernos emplea variantes sumamente eficientes:

- **Laravel (PHP) y Ruby on Rails (Ruby):** Ejemplifican la adaptación más ortodoxa orientada a servidor web. Un módulo enrutador mapea la URL de la petición entrante, llama a una función del Controlador, el cual extrae los registros interactuando con el Modelo (mediante el mapeador de objetos ORM como Eloquent o ActiveRecord), inyectando el estado final sobre plantillas gráficas nativas (Blade o ERB).
- **Django (Python):** Sigue exactamente el mismo razonamiento conceptual pero adoptando la nomenclatura funcional de **MVT (Model-View-Template)**. En Django, el 'Modelo' enlaza los esquemas de bases de datos, la 'Vista' realiza operativamente las funciones asignadas al Controlador clásico, y la 'Plantilla' (Template) asume el rol de la capa Vista estándar del MVC tradicional.
- **Spring MVC (Java) y ASP.NET Core MVC (C#):** Extienden el patrón hacia la robustez corporativa a gran escala a través de controladores centrales masivos (*Front Controller* como el DispatcherServlet de Spring). Estos frameworks operan de forma híbrida: son capaces de inyectar datos en vistas tradicionales del lado del servidor o bien omitir las vistas HTML tradicionales para responder datos puros estructurados en formato REST API (JSON).

6.- Ejemplo Sencillo: Sistema de Registro de Estudiantes

El siguiente caso ilustra la segmentación lógica requerida para codificar un proceso básico de almacenamiento y alta dentro de una plataforma estudiantil universitaria, aislando los componentes de acuerdo al estándar formal de MVC:

Modelo lógico de datos (Student.js)

```
class Student {
  constructor(matricula, nombre, carrera, correo) {
    this.matricula = matricula;
    this.nombre = nombre;
    this.carrera = carrera;
    this.correo = correo;
  }
  static async save(data) {
    if (!data.correo.includes('@')) throw new Error('Correo inválido');
    return db.table('students').insert(data);
  }
}
```

Controlador de la petición (StudentController.js)

```
class StudentController {
  static async register(req, res) {
    try {

      await Student.save(req.body);
      return res.render('successView', { user: req.body.nombre });
    } catch (err) {
      return res.render('formView', { error: err.message });
    }
  }
}
```

Vista gráfica de la interfaz (formView.html)

```
<form action='/students/register' method='POST'>
  <input type='text' name='matricula' placeholder='Matrícula' />
  <input type='text' name='nombre' placeholder='Nombre' />
  <input type='text' name='carrera' placeholder='Carrera' />
  <input type='email' name='correo' placeholder='Correo' />
  <button type='submit'>Registrar</button>
</form>
```

7.- Conclusión

Internalizar metódicamente la filosofía estructural implícita en el patrón Modelo-Vista-Controlador antes de aproximarse al ecosistema operativo de los frameworks web comerciales modernos representa una clara demarcación entre la codificación empírica y la verdadera ingeniería de software. Las tecnologías contemporáneas ocultan gran parte de esta complejidad mediante abstracciones sofisticadas y automatizaciones de andamiaje. Sin una asimilación clara de la separación de responsabilidades, el programador corre el inminente riesgo de comprometer gravemente la arquitectura de la aplicación, intercalando lógica relacional en las interfaces visuales o creando controladores masivos e inmanejables. La comprensión de MVC dota al profesional de criterios universales de diseño aplicables a cualquier lenguaje o stack tecnológico.

Referencias Bibliográficas

Django Software Foundation. (2026). *Django FAQ: Databases and models (MVT architecture)*. <https://docs.djangoproject.com/>

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. Otwell, T. (2026). *Laravel Documentation: The Basics of MVC structure*. <https://laravel.com/docs>

Reenskaug, T. (1979). *Models-Views-Controllers*. Xerox PARC Technical Memo. <http://www.skanks.com/trygve/Architecture/MVC/index.html>

Sommerville, I. (2019). *Ingeniería de Software* (10.^a ed.). Pearson Educación.